Especificación del Proyecto

Procesos de Ingeniería del Software — curso 2026-2027

---

1. Objetivo del proyecto

El objetivo de este proyecto es diseñar e implementar, de forma individual, la estructura base de una aplicación tipo SaaS (Software as a Service), organizada en capas y con gestión de usuarios.

Este documento especifica el **Sprint 1**. El apartado 11.1 propone un reparto semanal orientativo del trabajo.

2. Organización del trabajo

Cada alumno es responsable único de su propio repositorio (GitHub) y de la totalidad del código que contiene. No hay reparto de autoría: todo lo que hay en tu repositorio es tu responsabilidad, independientemente de las herramientas que hayas usado para producirlo.

3. Arquitectura obligatoria

Las siguientes son las únicas restricciones de arquitectura del proyecto. Fuera de ellas, la elección de tecnología (lenguaje, frameworks, librerías de autenticación, base de datos, etc.) es libre, siempre que puedas justificar y defender tu elección.

3.1. Separación cliente-servidor

- El backend sirve el frontend (mismo origen). No se permite que el cliente acceda directamente a la base de datos ni ejecute lógica de negocio: toda esa lógica reside en el servidor.
- La comunicación entre cliente y servidor se realiza exclusivamente a través de una API.

3.2. Arquitectura en capas

- Backend: capa API, capa de lógica de negocio y capa de acceso a datos, claramente separadas en distintos módulos/archivos.
- Frontend: capa de presentación (interfaz GUI) y cliente de comunicación con el servidor, separadas entre sí.

> *Estas dos invariantes se comprobarán en la prueba de evaluación individual: se te podrá pedir en cualquier momento que señales dónde está la frontera entre capas de tu propio proyecto.*

4. Requisitos funcionales

4.1. Gestión de usuarios

- Registro de usuario local, con confirmación de cuenta por correo electrónico. Hasta que confirma su correo, la cuenta queda en estado *pendiente* y no puede iniciar sesión.
- Inicio de sesión local (email + contraseña).
- Inicio de sesión con al menos un proveedor externo de autenticación (OAuth).
- Cierre de sesión, invalidando correctamente el estado de la sesión.
- Mantenimiento de la sesión entre recargas de página: un usuario que ya inició sesión no debe tener que volver a autenticarse al refrescar.

4.2. Roles y consultas sobre usuarios

La aplicación distinguirá, como mínimo, dos roles: **usuario** y **administrador**.

- **Listar usuarios**: solo el administrador. El listado muestra, para cada usuario, al menos su email, su rol y su estado.
- **Comprobar si un usuario está activo**: solo el administrador. En este proyecto, un usuario está *activo* cuando ha confirmado su cuenta por correo (o se registró mediante OAuth) y no ha sido eliminado. Esta consulta no se refiere a si el usuario está conectado en este momento.
- **Eliminar usuario**: el administrador puede eliminar a cualquier usuario; un usuario normal solo puede eliminar su propia cuenta. Un usuario eliminado no puede volver a iniciar sesión.
- Estas restricciones se aplican **en el servidor**: ocultar un botón en la interfaz no basta.
- Cómo se crea el primer administrador (script de inicialización, variable de entorno, promoción manual en la base de datos, etc.) es decisión de diseño del alumno y deberá poder justificarla.

5. Requisitos no funcionales y seguridad

- Persistencia real de los datos en una base de datos (no en memoria): los datos deben sobrevivir a un reinicio del servidor.
- Ninguna clave, contraseña o secreto debe aparecer en el código fuente ni subirse al repositorio; deben gestionarse mediante variables de entorno o un gestor de secretos. El repositorio incluirá un archivo de ejemplo (por ejemplo `.env.example`) con los nombres de las variables, sin valores reales.
- Las contraseñas de usuario deben almacenarse con una función de *hash* segura para contraseñas (por ejemplo bcrypt o argon2), nunca en texto plano ni con cifrado reversible.
- Las rutas o funcionalidades que requieren un usuario autenticado deben rechazar peticiones sin sesión válida, y las reservadas al administrador deben rechazar peticiones de usuarios sin ese rol.
- La aplicación debe desplegarse en un proveedor cloud con capa gratuita y ser accesible mediante una URL pública.

6. Calidad y pruebas

- Pruebas **unitarias automatizadas** de la capa de lógica: **al menos 5**, sobre las reglas de gestión de usuarios (por ejemplo: registro con un email ya existente, inicio de sesión con una cuenta pendiente de confirmar, un usuario normal que intenta eliminar a otro). Al menos una debe comprobar un caso de error, no solo el camino feliz.
- Todas las pruebas deben ser **automatizadas**: escritas como código con un framework de pruebas, se ejecutan con un único comando documentado en el README y comprueban el resultado por sí mismas, sin intervención manual. No cuentan como pruebas las comprobaciones hechas a mano en el navegador o con herramientas como Postman sin aserciones automatizadas.
- Manejo explícito de errores y casos excepcionales, con feedback claro al usuario.
- Registro (log) de la actividad relevante del sistema.

El control de versiones debe reflejar trabajo incremental: se espera un historial de commits con mensajes representativos del progreso, no una única entrega masiva.

7. Flujo de trabajo y pipeline de CI/CD

La asignatura trata sobre procesos, así que no basta con que la aplicación funcione: también se evalúa **cómo llega cada cambio desde tu editor hasta producción**. En ese camino se distinguen dos partes:

- el **flujo de trabajo**: los pasos que das tú (crear una rama, programar, abrir un *pull request*, integrarlo);
- el **pipeline de CI/CD**: la parte automatizada que se dispara con cada cambio (ejecutar las pruebas y desplegar). CI es *integración continua*; CD, en este proyecto, es *despliegue continuo*: cada cambio integrado llega a producción sin intervención manual.

7.1. Flujo de trabajo

Se seguirá el modelo conocido como **GitHub Flow**:

- **Rama por cambio.** Cada funcionalidad o corrección se desarrolla en una rama propia. No se trabaja directamente sobre la rama principal (`main`).
- **Pruebas en local.** Antes de abrir el *pull request*, las pruebas automatizadas del apartado 6 se ejecutan en local con el comando documentado en el README.
- **Integración mediante pull request.** Cada rama se incorpora a `main` mediante un *pull request*, aunque lo revises y aceptes tú mismo.

7.2. Automatización

- **Integración continua.** Un servicio de CI (por ejemplo, GitHub Actions) ejecuta automáticamente las pruebas en cada *pull request* y en cada cambio en `main`. No se integra un *pull request* con el CI en rojo.
- **Despliegue continuo.** Cada cambio que llega a `main` se despliega automáticamente en la URL pública, ya sea mediante la integración del proveedor cloud con el repositorio o mediante un paso del propio CI. No se despliega a mano.

No se exige que el despliegue espere al resultado del CI: ambos pueden funcionar de forma independiente.

La configuración del CI y del despliegue forma parte del repositorio y del código que debes saber explicar. El historial de ramas, *pull requests* y ejecuciones del CI es la evidencia de que se ha seguido este flujo y podrá usarse en la prueba individual (por ejemplo: «¿por qué falló esta ejecución y cómo lo resolviste?»).

> *En GitHub gratuito, la protección de ramas (impedir push directo a `main` o exigir el CI en verde) solo está disponible en repositorios públicos (o en privados con GitHub Pro, gratuito para estudiantes mediante GitHub Education). Si no puedes activarla, el flujo se exige igualmente: se comprobará en el historial.*

8. Entrega del Sprint 1

Fecha límite: en la fecha fijada en la herramienta de gestión.

Se entregan tres cosas:

- **URL del repositorio** (GitHub.
- **URL pública** de la aplicación desplegada, funcionando en la fecha de entrega y durante el periodo de evaluación.
- **README** en la raíz del repositorio que incluya: tecnologías elegidas y una frase que justifique cada una; cómo ejecutar el proyecto en local; cómo ejecutar las pruebas; variables de entorno necesarias (solo nombres); y cómo acceder como administrador para la revisión (sin publicar contraseñas en el repositorio: las credenciales de prueba se envían junto con la entrega).

Se tomará como entregado el último commit de la rama principal anterior a la fecha límite.

9. Avisos prácticos

No forman parte de los requisitos, pero son los puntos donde es más fácil atascarse. Conviene resolverlos pronto, no la última semana.

- **Envío de correo.** Los proveedores cloud gratuitos suelen bloquear el envío directo por SMTP. Lo habitual es usar un servicio de correo transaccional con capa gratuita (API o SMTP autenticado). Revisa también la carpeta de spam al probar la confirmación de cuenta.

- **Callback de OAuth.** El proveedor externo solo acepta las URL de retorno que registres. Necesitarás registrar tanto la de desarrollo (localhost) como la de la URL pública desplegada, y la configuración debe coincidir exactamente (protocolo, dominio y ruta).
- **Capas gratuitas.** Algunos servicios suspenden la aplicación tras un periodo de inactividad y tardan en arrancar en la primera petición. Tenlo en cuenta el día de la entrega y de la prueba.

10. Calendario

El **Sprint 1** dura un mes y cubre íntegramente los apartados 1 a 8 de este documento.

Hitos semanales del Sprint 1 (orientativos)

Para que el trabajo del mes no se acumule al final, se propone repartirlo en cuatro hitos, uno por semana. Los hitos son **orientativos**: sirven para organizarte y no se entregan ni se califican por separado. Lo que cuenta es la entrega del Sprint 1 completa (apartado 8).

Hito 1 (semana 1). Esqueleto y backend.

- Repositorio creado y flujo de trabajo con ramas y *pull requests* (apartado 7.1).
- Integración continua ejecutando las pruebas y despliegue continuo en la URL pública desde el primer día, aunque al principio solo muestre una página mínima (apartado 7.2).
- Capa lógica de gestión de usuarios con los datos **en memoria** (alta, listado, comprobar si está activo, eliminar) y sus pruebas unitarias automatizadas (apartado 6).
- API que expone esa lógica (capa de presentación del backend).

Hito 2 (semana 2). Frontend y sesión.

- Interfaz y capa de comunicación del cliente, separadas entre sí (apartado 3.2).
- Registro e inicio de sesión locales, con la contraseña almacenada con *hash*; cierre de sesión y mantenimiento de la sesión entre recargas (apartado 4.1). Los datos siguen en memoria.
- Rutas protegidas que rechazan peticiones sin sesión válida (apartado 5).

Hito 3 (semana 3). Persistencia y roles.

- Capa de acceso a datos contra una base de datos real, de modo que los datos sobrevivan a un reinicio (apartado 5). El objetivo es **sustituir la implementación en memoria sin modificar la capa lógica**: si tienes que cambiar la lógica para introducir la base de datos, revisa la separación entre capas.
- Secretos (cadena de conexión, claves) fuera del código, en variables de entorno (apartado 5).
- Roles de usuario y administrador, con las restricciones del apartado 4.2 aplicadas en el servidor.
- Registro (log) de la actividad relevante (apartado 6).

Hito 4 (semana 4). Servicios externos y cierre.

- Inicio de sesión con un proveedor externo mediante OAuth (apartado 4.1).
- Confirmación de cuenta por correo electrónico (apartado 4.1). Ambas tareas requieren configurar un servicio externo con la URL pública: conviene empezar por ahí.
- Manejo de errores con *feedback* claro al usuario, README completo y revisión final de la entrega.

> *Recomendación: al terminar cada hito, marca el último commit de la rama principal con una etiqueta (por ejemplo, `hito-1`) comprobando que el CI está en verde y la aplicación desplegada funciona. Te servirá para ver tu progreso y para explicar la evolución del proyecto en la prueba individual.*